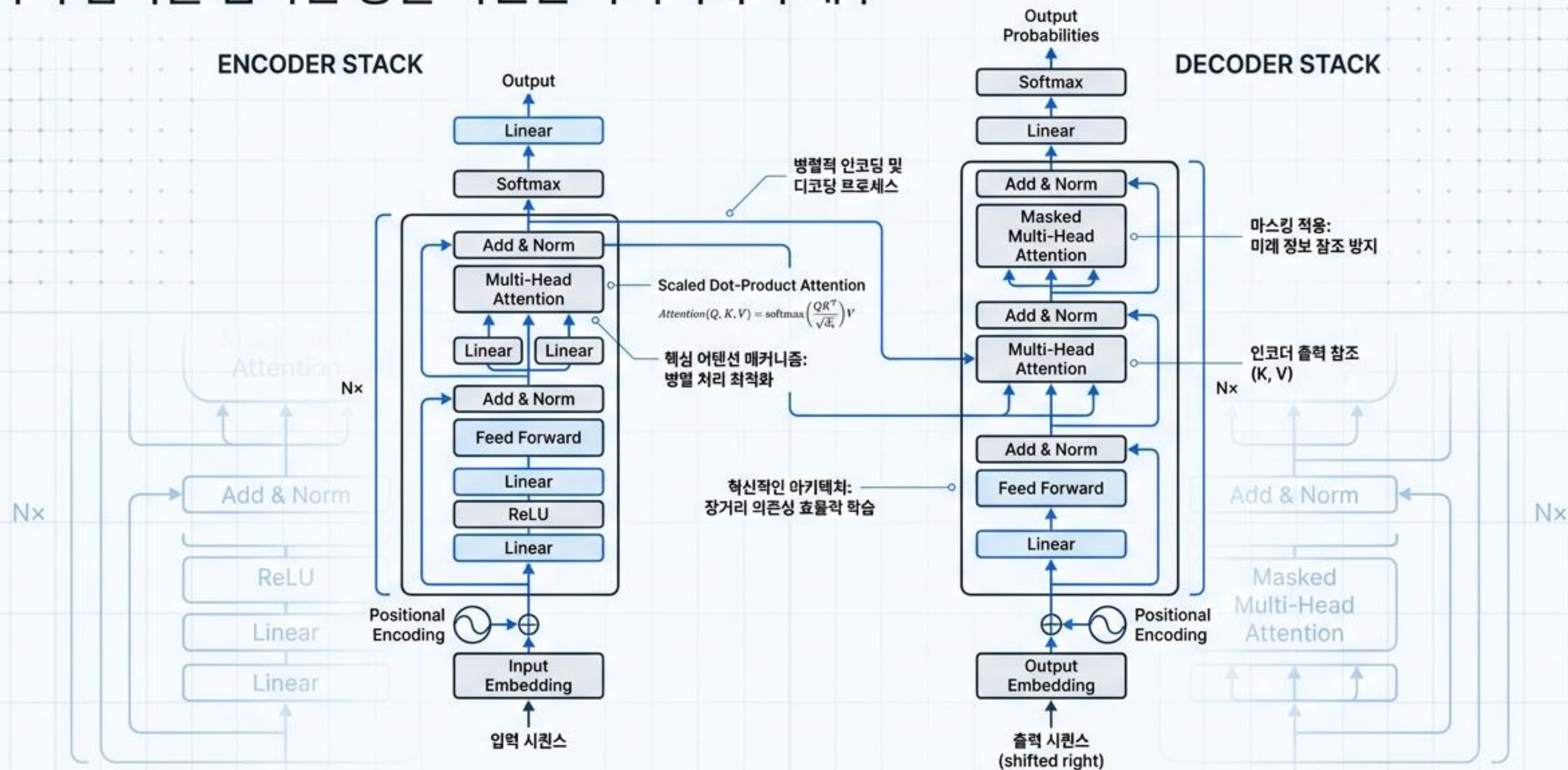


트랜스포머 완전 정복: Attention is all you need

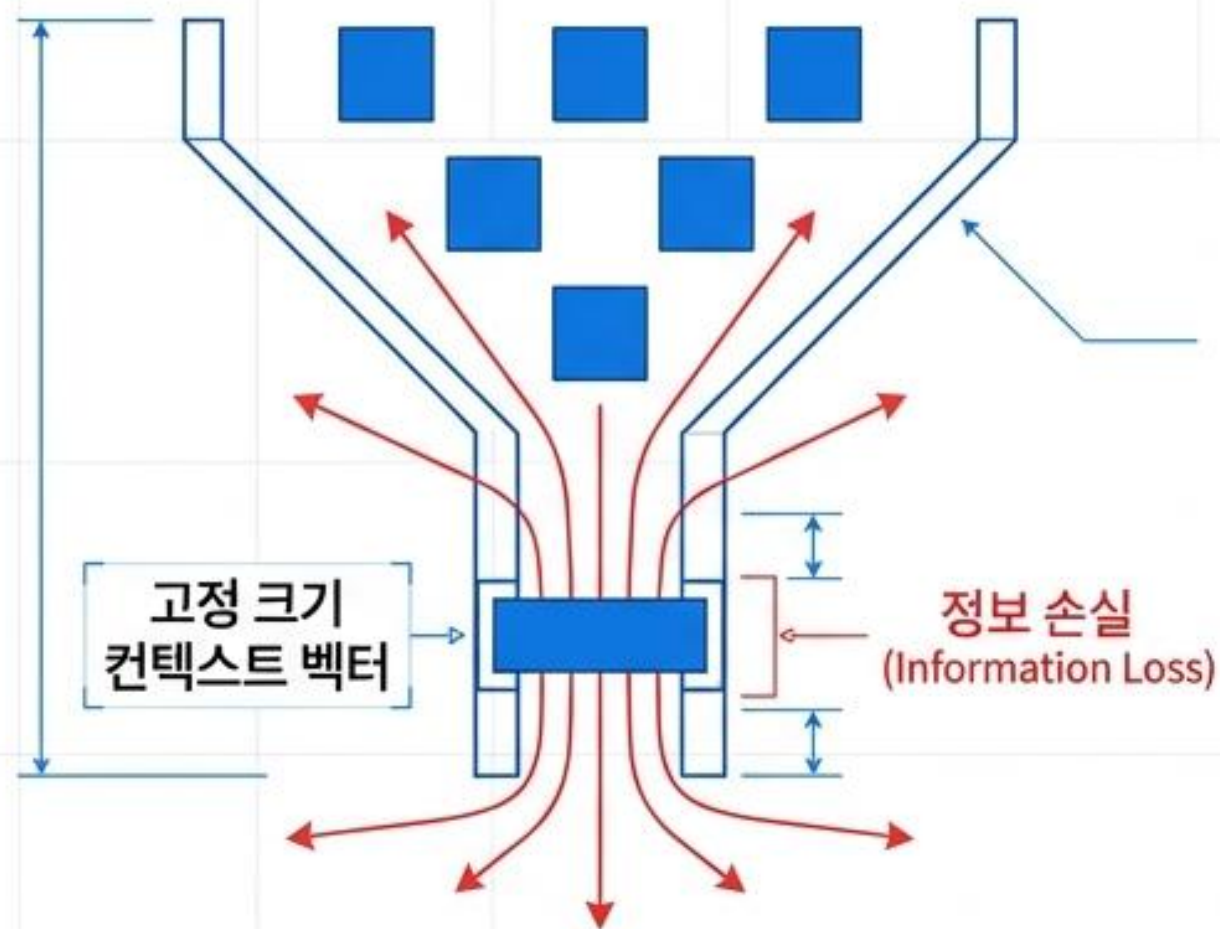
순차적 압축을 넘어선 병렬 어텐션 아키텍처의 해부



RNN의 한계를 극복한 새로운 설계 패러다임

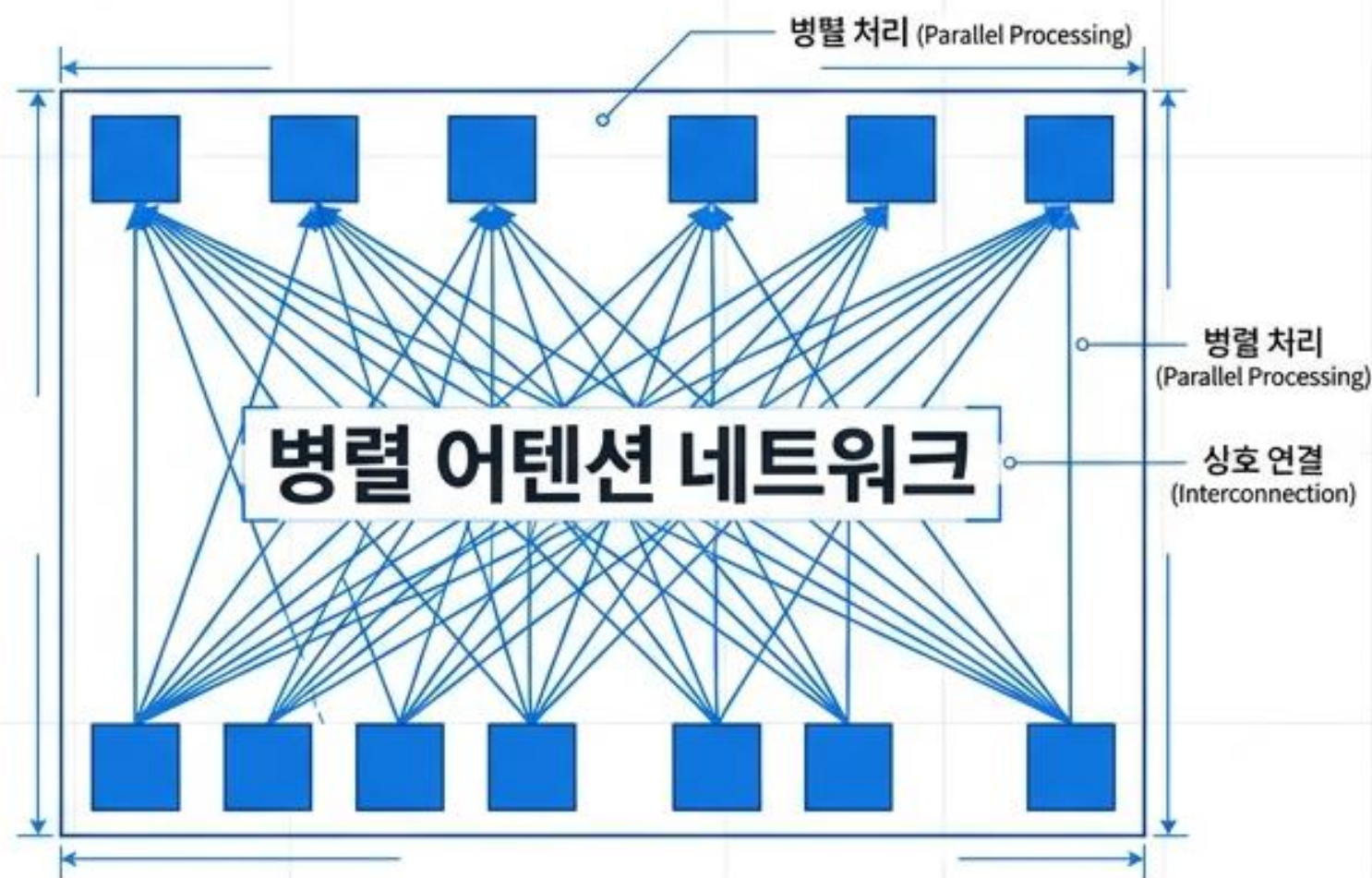
기존 seq2seq 병목 구조

- 인코더가 입력 시퀀스를 고정된 크기의 단일 벡터로 압축
- 필연적인 문맥 정보 손실 발생
- 어텐션을 단지 정보 손실 보정용 보조 도구로 사용



트랜스포머 병렬 구조

- RNN 연산을 완전히 배제한 혁신적 접근
- 어텐션(Attention) 메커니즘만으로 아키텍처 완벽 재구성
- 모든 입력 데이터가 상호 연결되는 무손실 병렬 처리



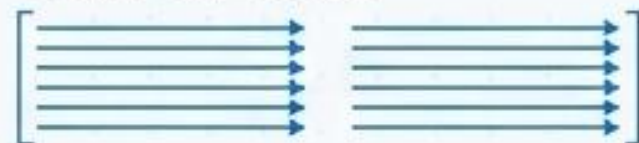
트랜스포머 설계도: 거시적 구조와 핵심 규격

STRUCTURAL HYPERPARAMETERS

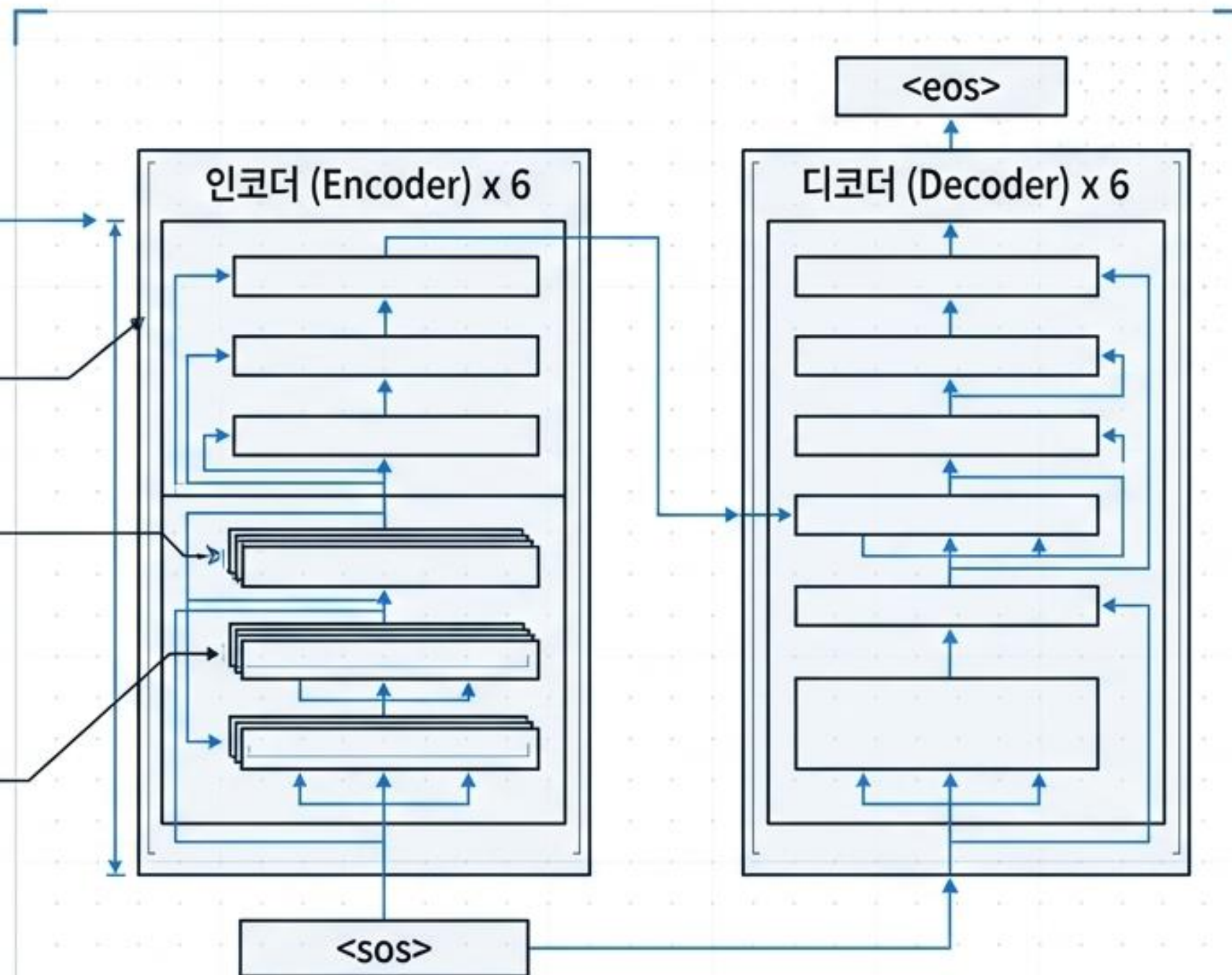
$d_{\text{model}} = 512$: 모델 내 모든 층의 고정된 입출력 차원
(임베딩 벡터 차원과 동일)

$\text{num_layers} = 6$: 인코더와 디코더가 각각
스택(Stack)된 층의 개수

$\text{num_heads} = 8$: 병렬 처리를 위해 분할된
어텐션 헤드의 개수



$d_{\text{ff}} = 2048$: 내부 피드 포워드 신경망(FFNN)의
은닉층 크기



위치 정보의 주입: 포지셔널 인코딩 (Positional Encoding)

- 순차적 입력의 부재: RNN과 달리 단어를 병렬로 한 번에 입력받으므로, 단어의 위치(순서) 정보가 별도로 필요
- 수학적 위치 정보 생성: 사인(sin)과 코사인(cos) 함수의 주기성을 활용하여 각 위치마다 고유한 패턴 생성
- 변수 정의: pos는 문장 내 단어의 위치, i는 임베딩 벡터 내 차원의 인덱스
- 결합 연산: (문장 임베딩 벡터) + (포지셔널 인코딩 벡터)

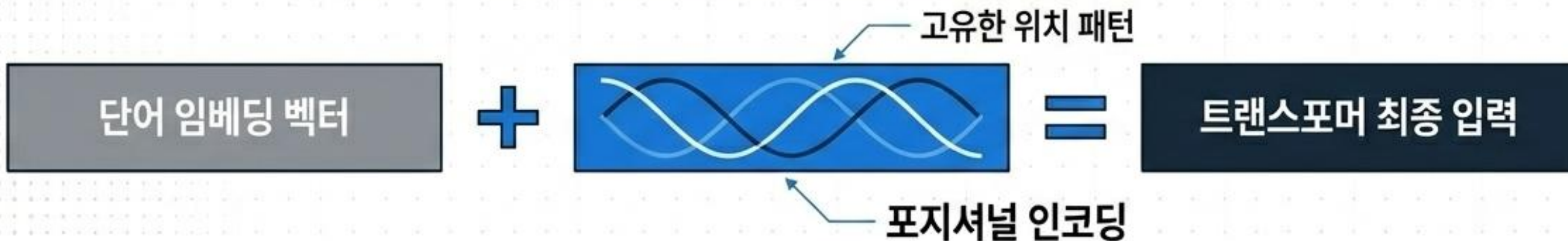
$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

위치 인덱스

차원 인덱스

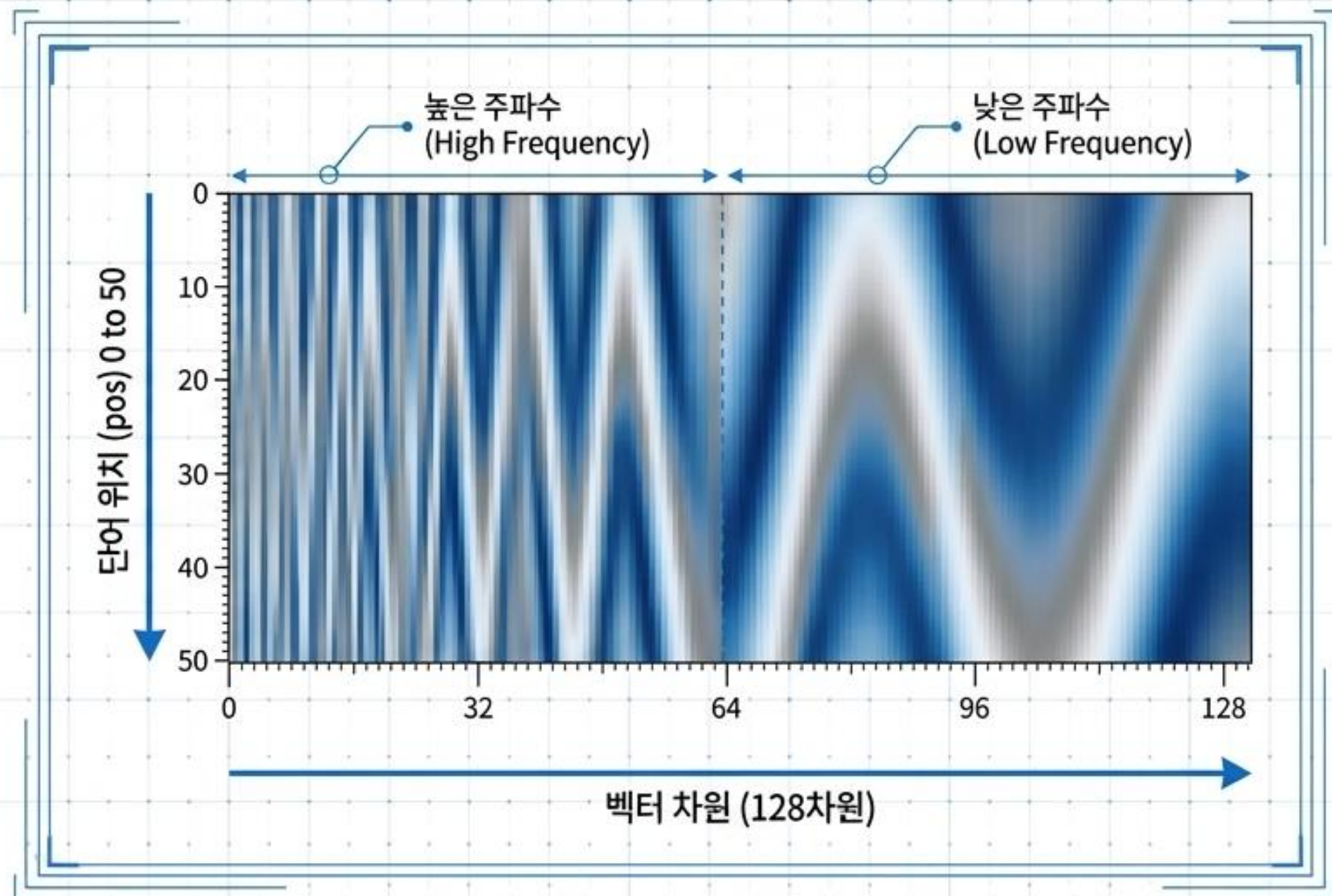
임베딩 차원



포지셔널 인코딩 행렬 시각화

미니맵: 📍 데이터 입력층 (Input & Positional Encoding)

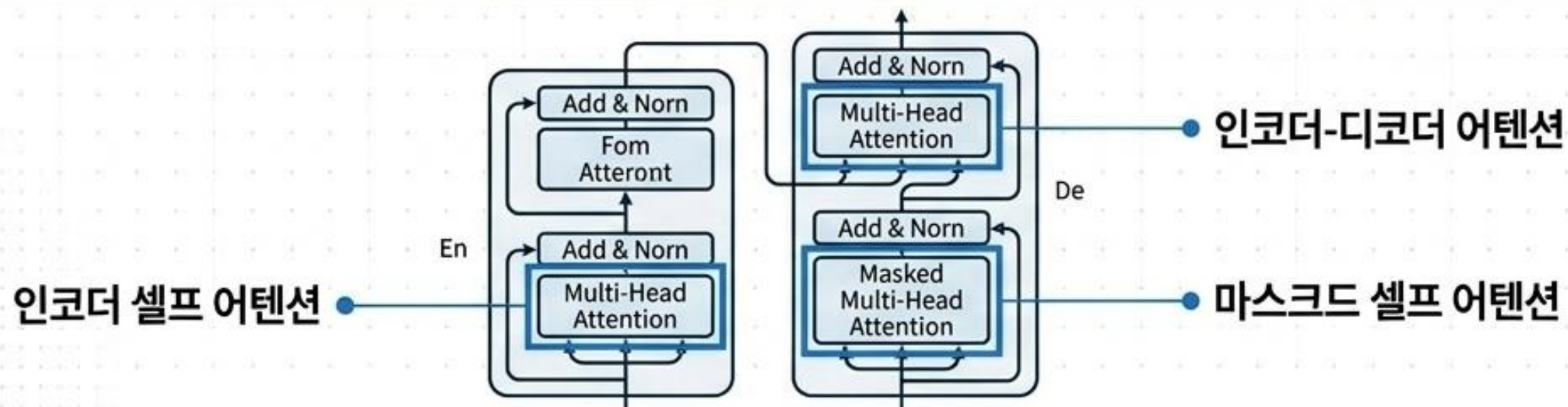
- **순서 정보의 보존**: 같은 단어라도 문장 내 위치(pos)에 따라 트랜스포머에 입력되는 최종 벡터값이 달라짐
- **행렬 스펙 예시**: 50개의 단어로 구성된 문장, 128차원의 임베딩 크기를 가정할 때 생성되는 50 x 128 크기의 행렬
- 짝수 인덱스는 사인 함수, 홀수 인덱스는 코사인 함수 값을 가지며 고유한 그래픽 패턴을 형성



트랜스포머를 구동하는 3가지 핵심 어텐션

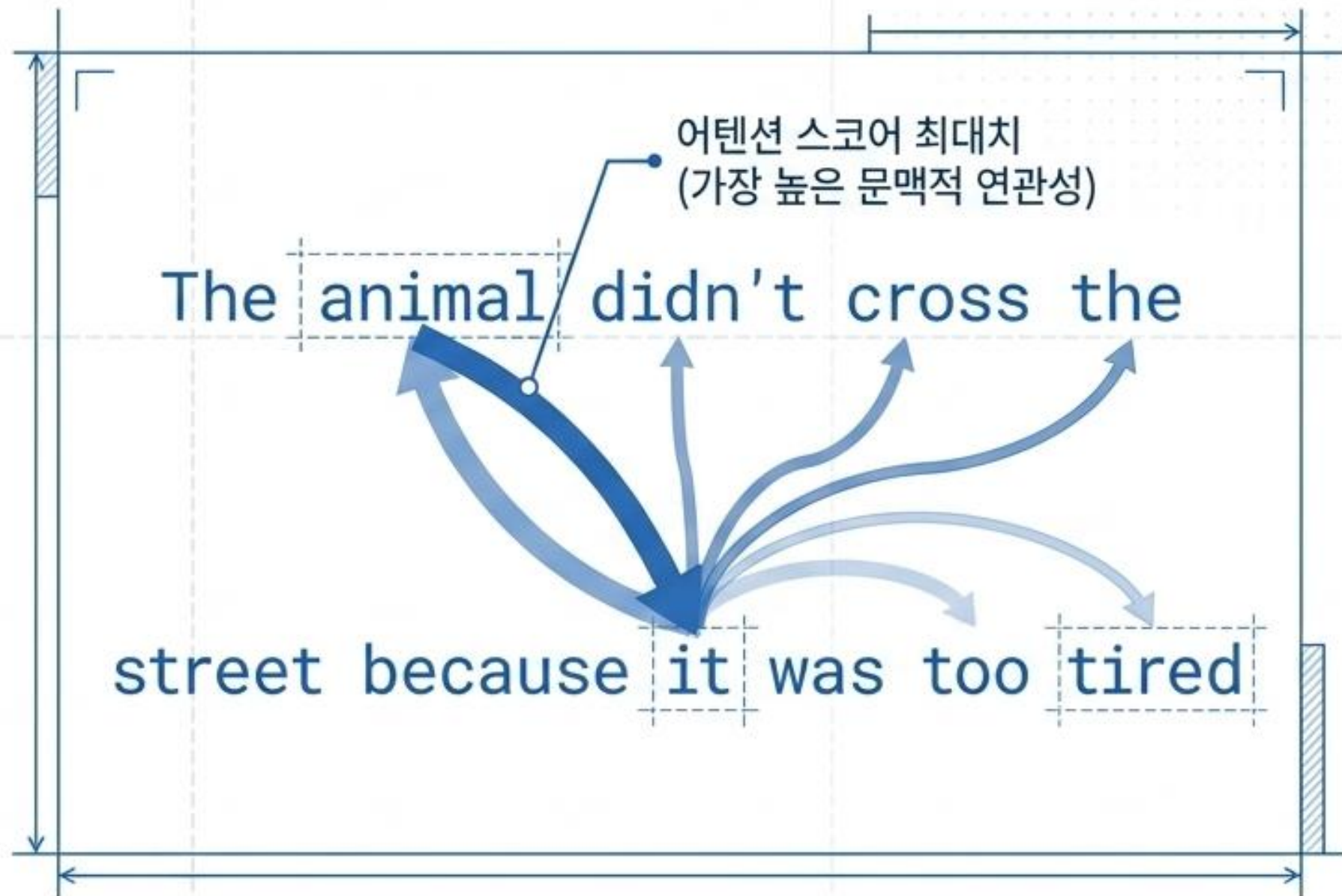
- 어텐션의 종류는 Query(Q), Key(K), Value(V) 벡터의 출처(Origin)가 어디인지에 따라 구분
- 셀프 어텐션(Self-Attention)은 Q, K, V가 모두 동일한 출처에서 파생됨을 의미

어텐션 종류	위치	Query (Q)	Key (K)	Value (V)
인코더 셀프 어텐션	인코더	인코더 벡터	인코더 벡터	인코더 벡터
마스크드 셀프 어텐션	디코더	디코더 벡터	디코더 벡터	디코더 벡터
인코더-디코더 어텐션	디코더	디코더 벡터	인코더 벡터	인코더 벡터



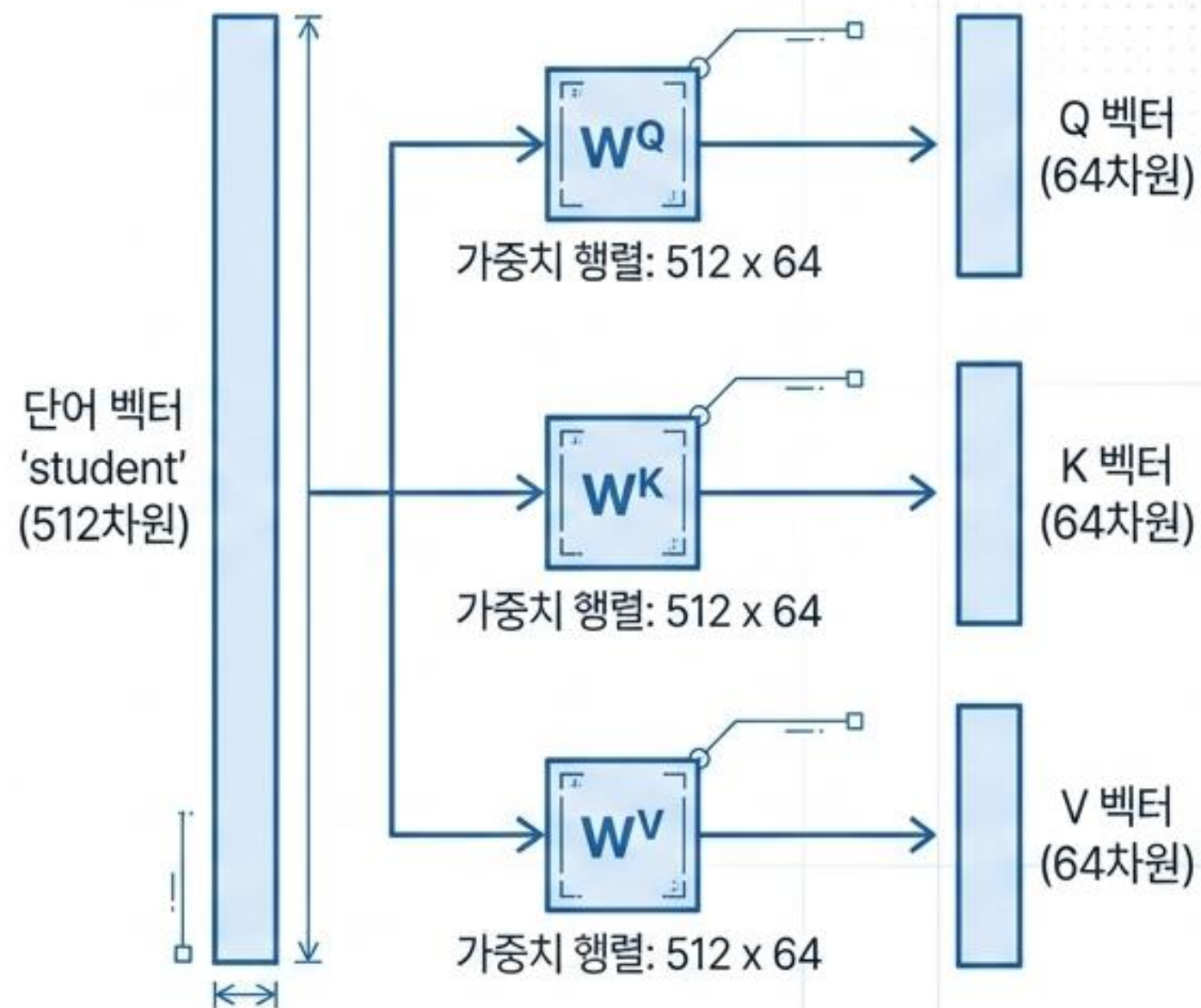
셀프 어텐션(Self-Attention): 문맥의 실마리 찾기

- **구조적 위치:** N개의 인코더 층 내부의 첫 번째 서브층
- **개념:** 입력 문장 내의 단어들끼리 상호 간의 유사도를 구하여 문맥적 의미를 파악
- **효과 증명:** 기계는 대명사 'it'이 가리키는 대상을 알기 어렵지만, 셀프 어텐션을 통해 'animal'과의 연관도가 높음을 스스로 학습



Q, K, V 벡터의 생성 과정

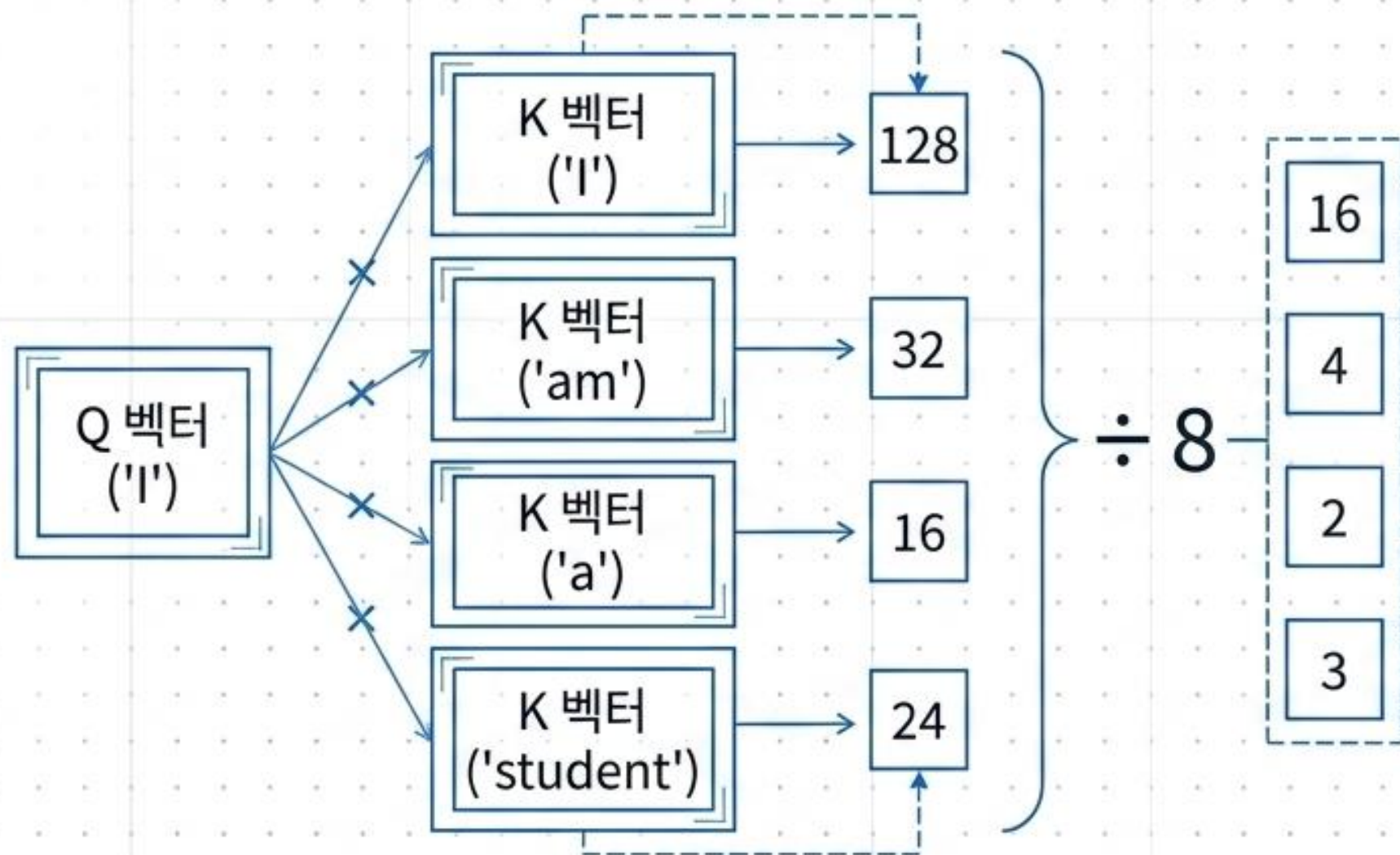
- **입력 벡터의 차원 축소:** d_{model} (512차원)
단어 벡터를 직접 사용하지 않고 분할
- **차원 축소 공식:** $d_{\text{model}} / \text{num_heads}$
연산에 따라 $512 / 8 = 64$ 차원의 벡터 생성
- **가중치 행렬 연산:** 각 단어 벡터에 훈련 가능한 3개의 가중치 행렬(W^Q, W^K, W^V)을 곱하여 도출





스케일드 닷-프로덕트 계산 1단계: 스코어 산출

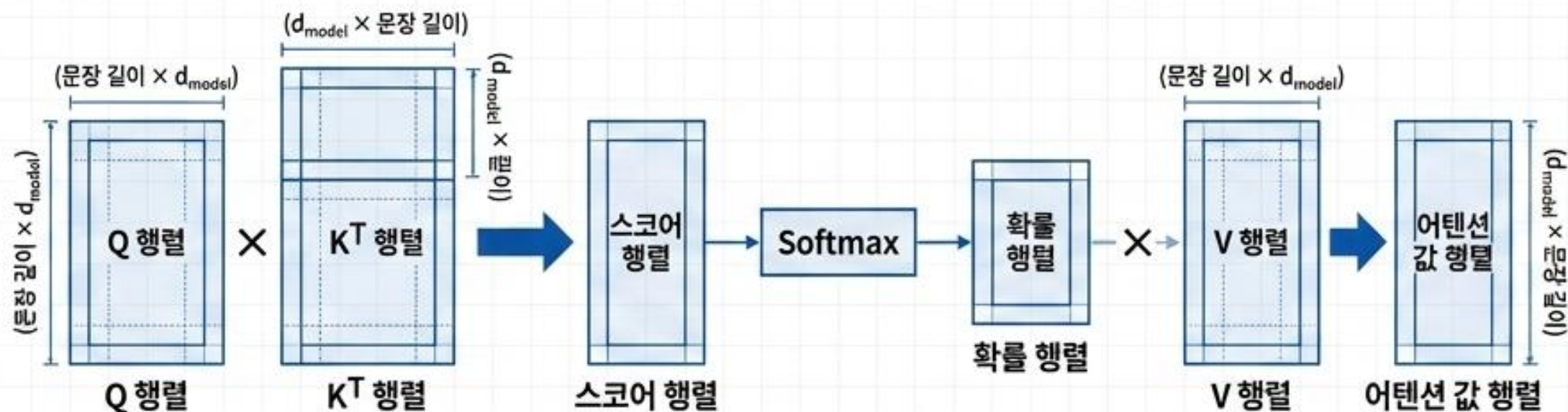
- 어텐션 스코어 계산: 특정 단어의 Query(Q) 벡터와 모든 단어의 Key(K) 벡터를 내적(Dot-product)하여 연관도 측정
- 스케일링(Scaling): 내적 값을 루트(d_k) (64의 제곱근인 8)로 나누어 값의 발산을 방지
- 연산 예시: 단어 'I'의 Q벡터를 'I', 'am', 'a', 'student'의 모든 K벡터와 내적하여 스코어 도출



스케일드 닷-프로덕트 계산 2단계: 행렬 일괄 처리

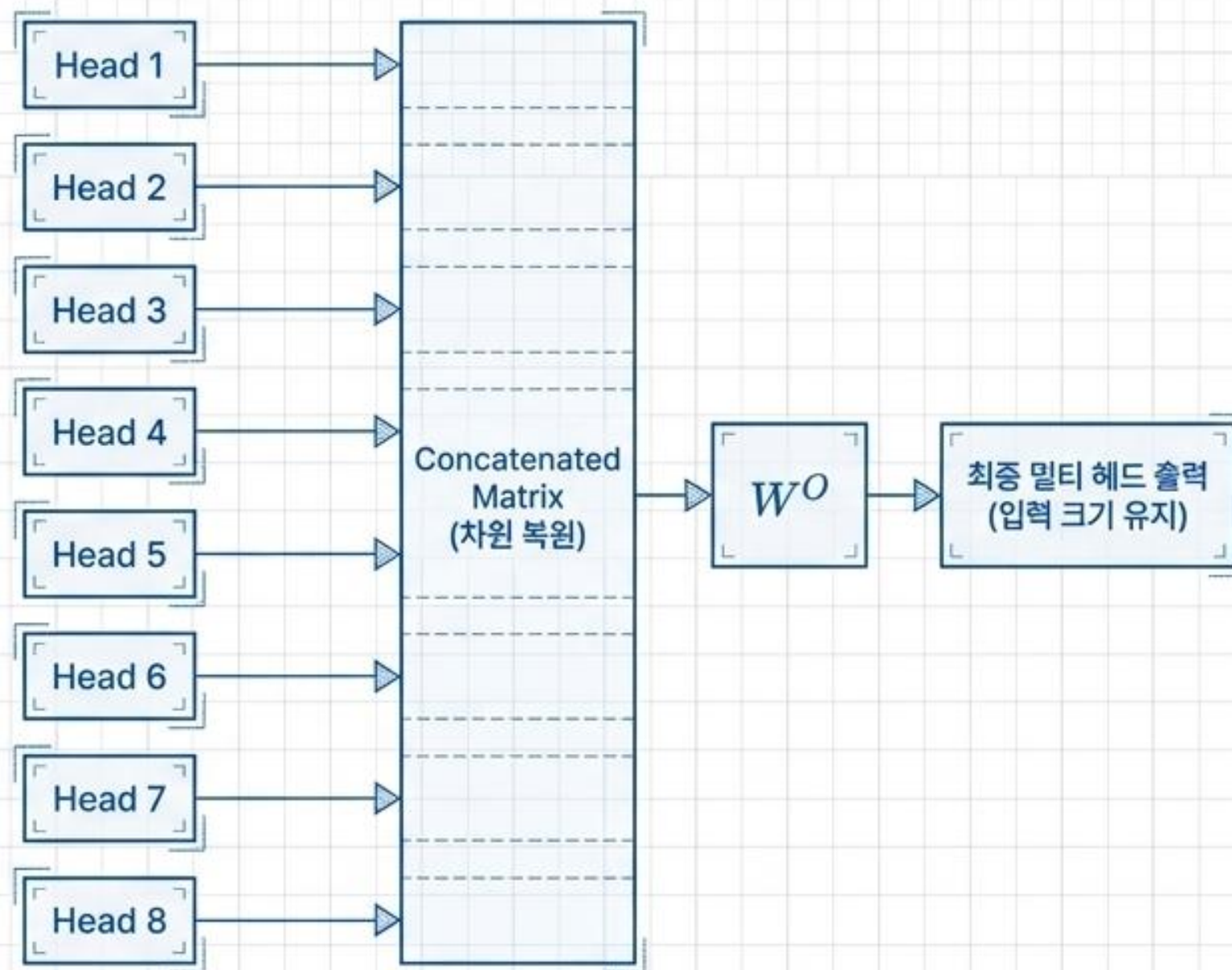
- 개별 벡터 연산이 아닌,
문장 전체를 행렬(Matrix)로
구성하여 병렬 일괄 계산 수행
- 소프트맥스(Softmax) 적용:
산출된 어텐션 스코어 행렬에
소프트맥스를 취해
확률값(어텐션 분포) 생성
- 최종 어텐션 값 도출:
확률값 행렬에 Value(V) 행렬을
곱하여 각 단어의 최종 컨텍스트
벡터 도출

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q} \cdot \mathbf{K}^T}{\sqrt{d_k}} \right) \cdot \mathbf{V}$$



멀티 헤드 어텐션(Multi-Head Attention): 다각도 시각의 통합

- **히드라 메타포:** 8개의 어텐션 헤드를 통해 문맥을 다각도로 분석 (예: 주어 동사 관계, 수식어 관계 등 분산 학습)
- **독립 연산:** num_heads = 8 설정에 따라 8개의 독립적인 스케일드 닷-프로덕트 어텐션을 병렬로 수행
- **행렬 병합(Concatenate):** 분할되었던 8개의 어텐션 결과를 하나로 길게 연결하여 원래 차원(d_model) 복원
- **최종 선형 변환:** 가중치 행렬 W^O 를 곱해 멀티 헤드 어텐션의 최종 결과물 산출

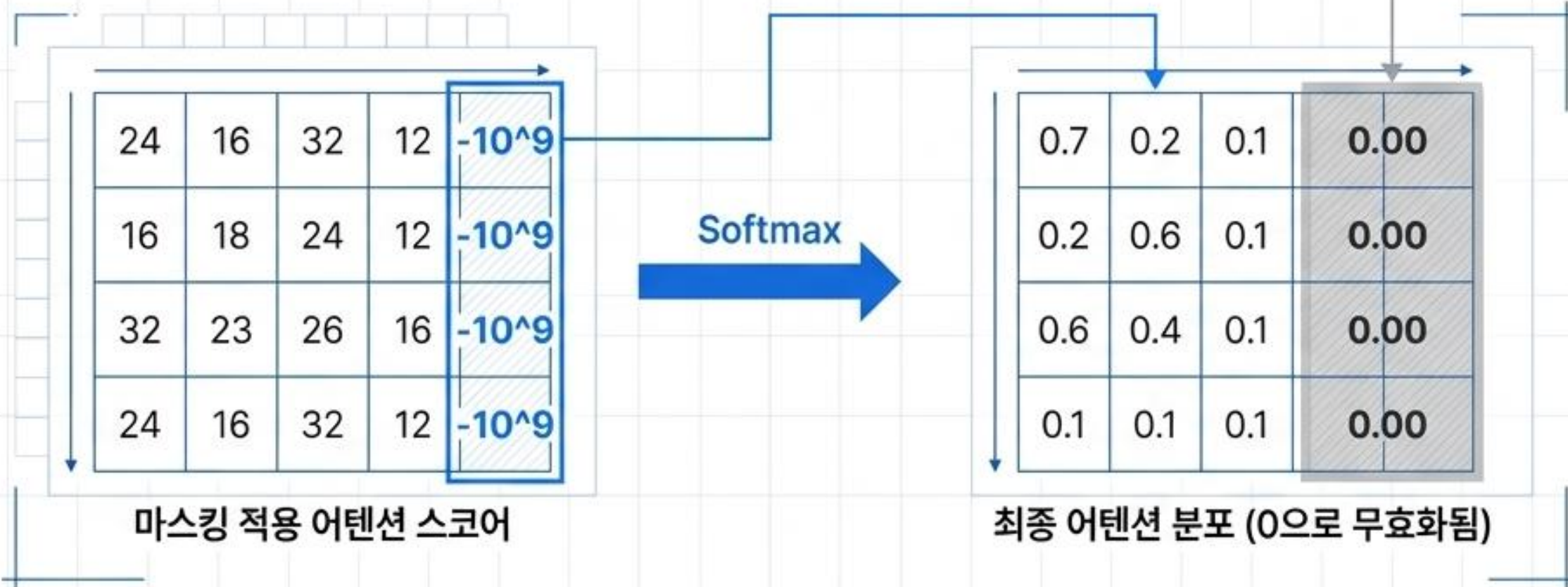




무의미한 정보 차단: 패딩 마스크(Padding Mask)

- 입력 시퀀스의 길이를 맞추기 위해 추가된 <PAD> 토큰은 실질적인 의미가 없으므로 어텐션 연산에서 제외 필수
- 마스크링 원리: Key 행렬에 <PAD>가 존재하는 열(Column) 전체에 매우 작은 음수값 ($-1,000,000,000$) 부여
- 소프트맥스 효과: 음수값이 부여된 위치는 소프트맥스 함수 통과 후 사실상 '0'이 되어 유사도 연산에서 완벽히 배제됨

Before and After Softmax

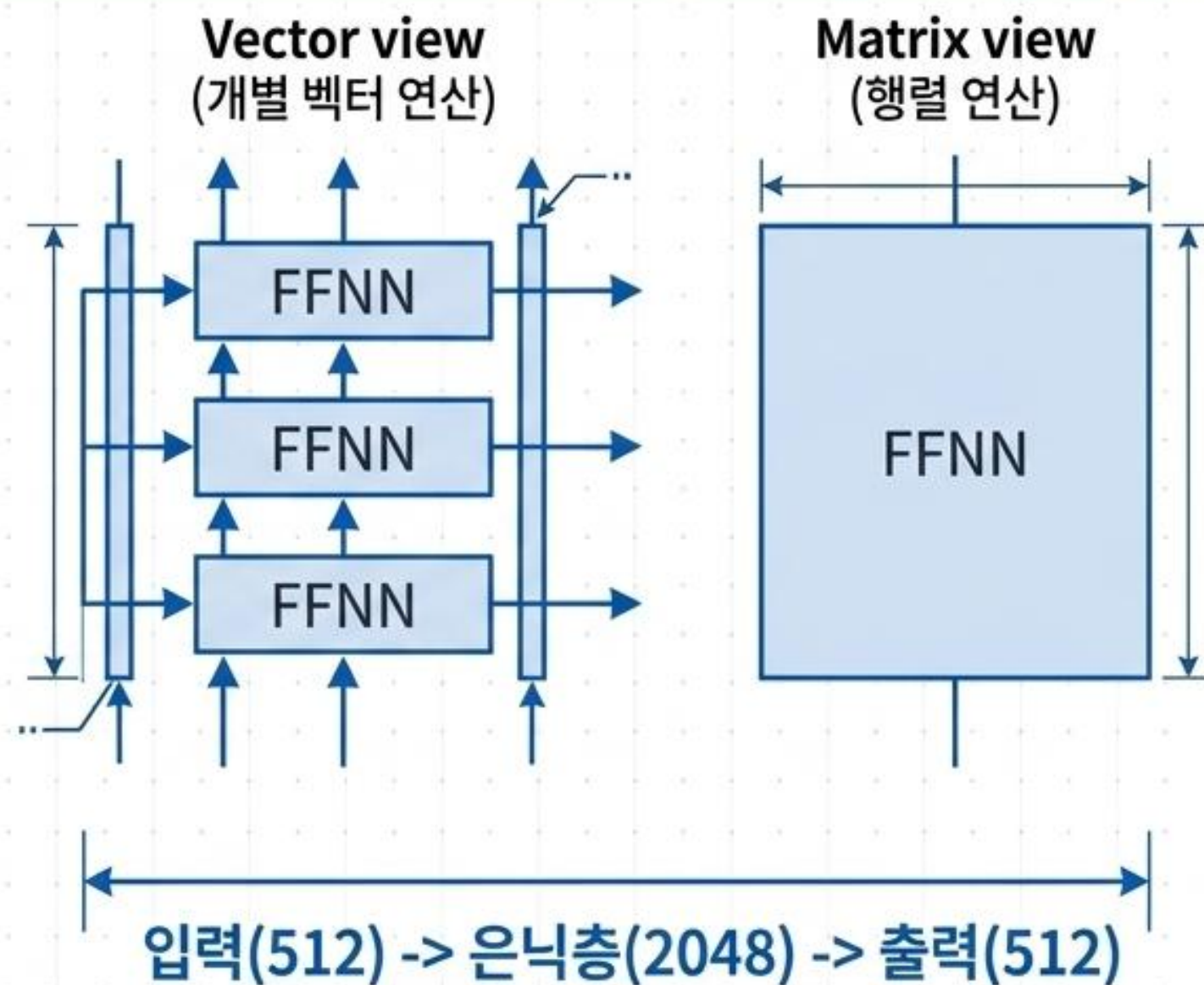


포지션-와이즈 피드 포워드 신경망

- 구조적 위치: 인코더와 디코더 모두에 공통적으로 존재하는 두 번째 서브층
- 완전 연결(Fully-connected) 구조: 개별 단어 벡터마다 독립적으로 동일한 연산 적용
- 차원의 팽창과 수축: 은닉층의 크기(d_{ff})는 2048차원으로 확장되었다가 다시 512차원으로 축소됨
- 출력 보존: 최종 출력은 입력 시의 크기(seq_len, d_model)를 완벽히 보존

Math Box

$$FFNN(\mathbf{x}) = \text{MAX}(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$



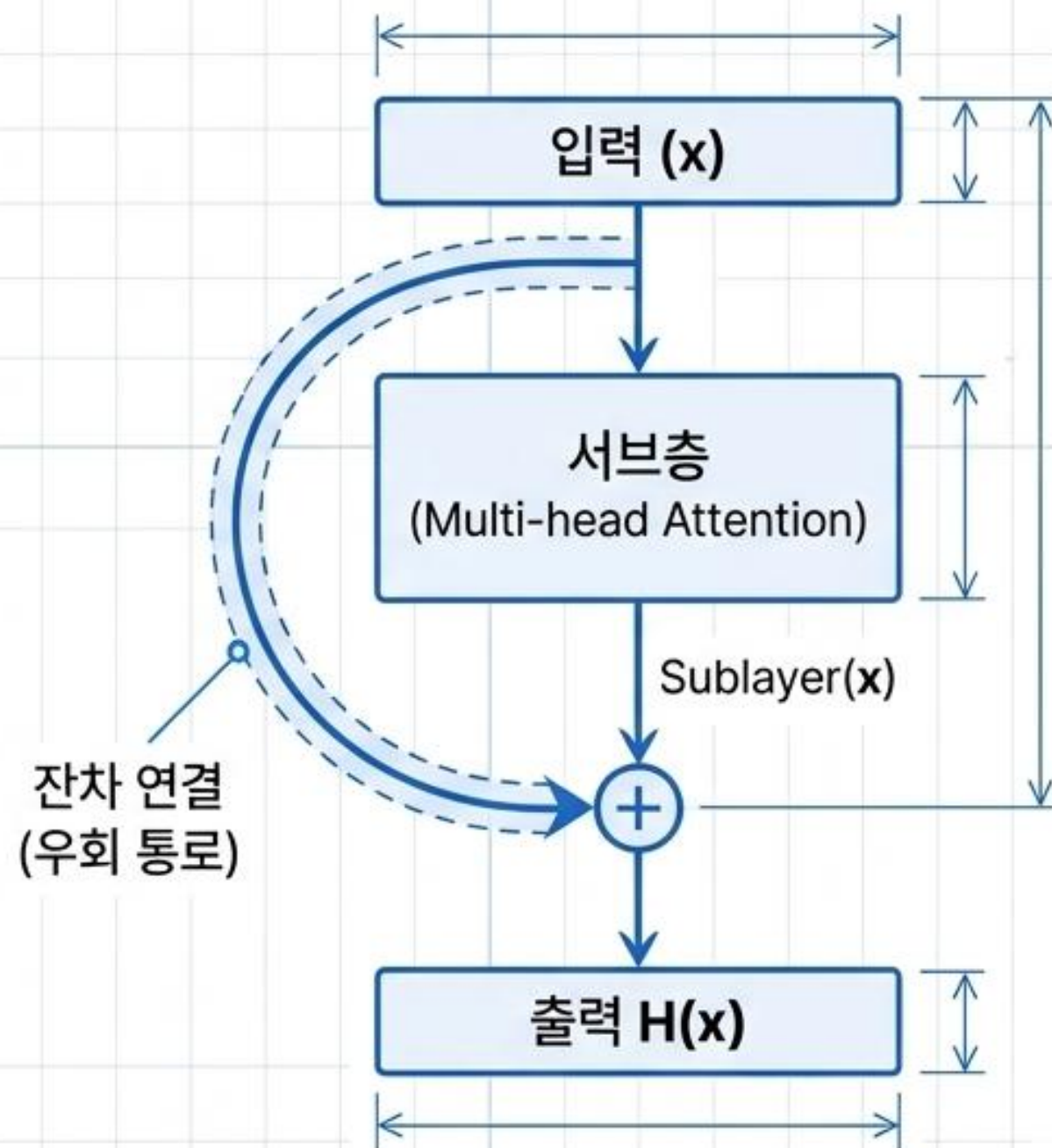
학습 안정성 강화: Add (잔차 연결)

- **딥러닝의 고질적 한계 극복:** 모델이 깊어질수록 발생할 수 있는 학습 손실(기울기 소실) 문제 방지
- **직결 통로 (우회로):** 서브층(어텐션 또는 FFNN)의 입력 행렬을 연산 결과 행렬에 그대로 더해줌
- **동일 차원의 조건:** 입력과 출력의 차원이 d_{model} 로 동일하게 설계되어 있기 때문에 단순 행렬 덧셈이 가능

Math Box

$$H(x) = x + \text{Sublayer}(x)$$

$$\text{예) } H(x) = x + \text{Multi-head_Attention}(x)$$



데이터 스케일 정렬: Norm (층 정규화)

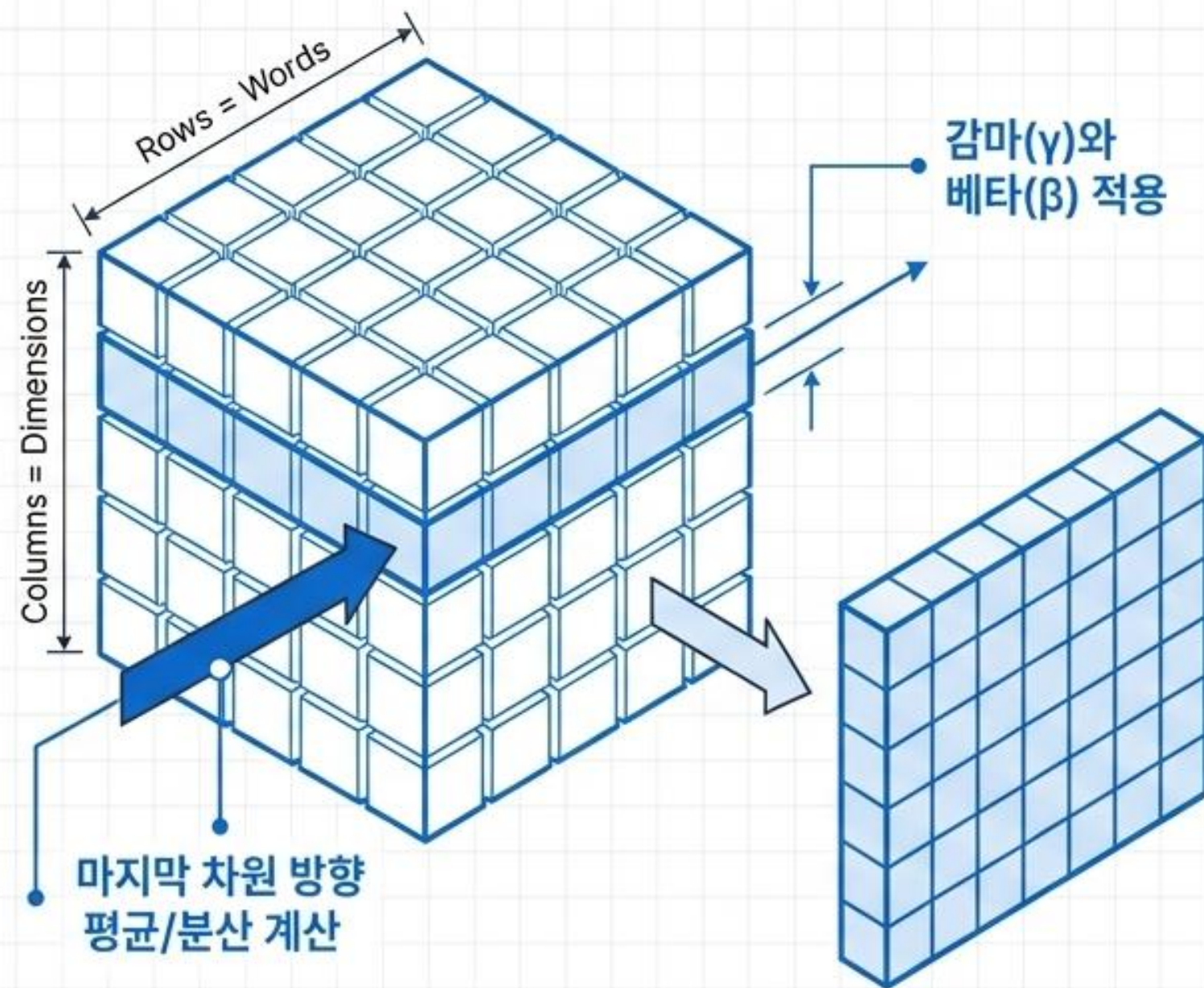
- **분포 균일화:** 잔차 연결을 거친 결과 행렬의 데이터 분포 편차를 정규화하여 학습 안정화
- **정규화 방향:** 텐서의 마지막 차원인 d_{model} (512차원) 축을 따라 각 단어 벡터별로 평균(μ)과 분산(σ^2) 계산
- **학습 파라미터:** 모델이 스스로 최적화할 수 있도록 감마(γ)와 베타(β) 파라미터 도입

Math Box (기본 정규화)

$$\hat{x}_i = \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}$$

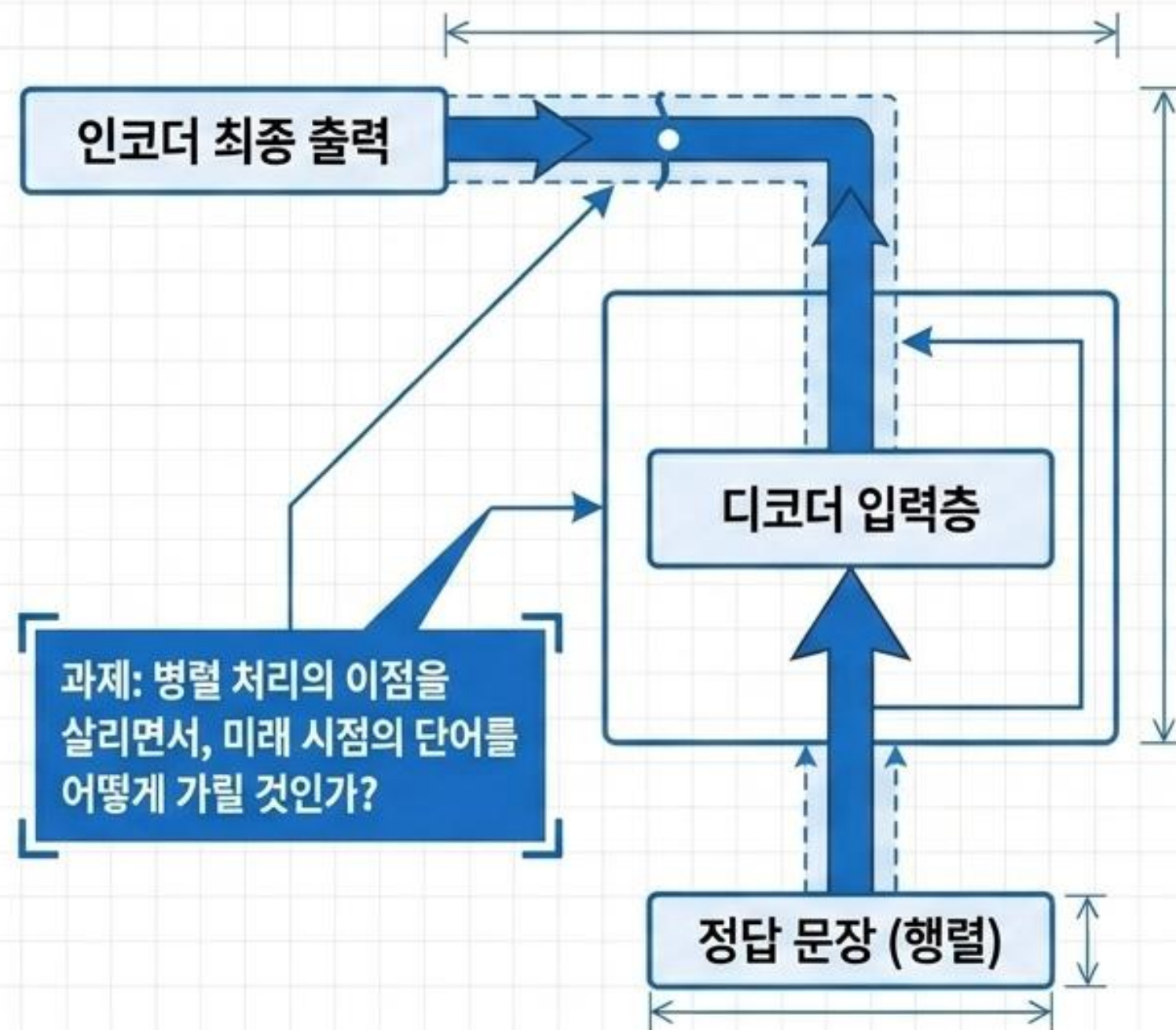
Math Box (스케일 및 시프트)

$$\text{ln}_i = \gamma \hat{x}_i + \beta = \text{LayerNorm}(x_i)$$



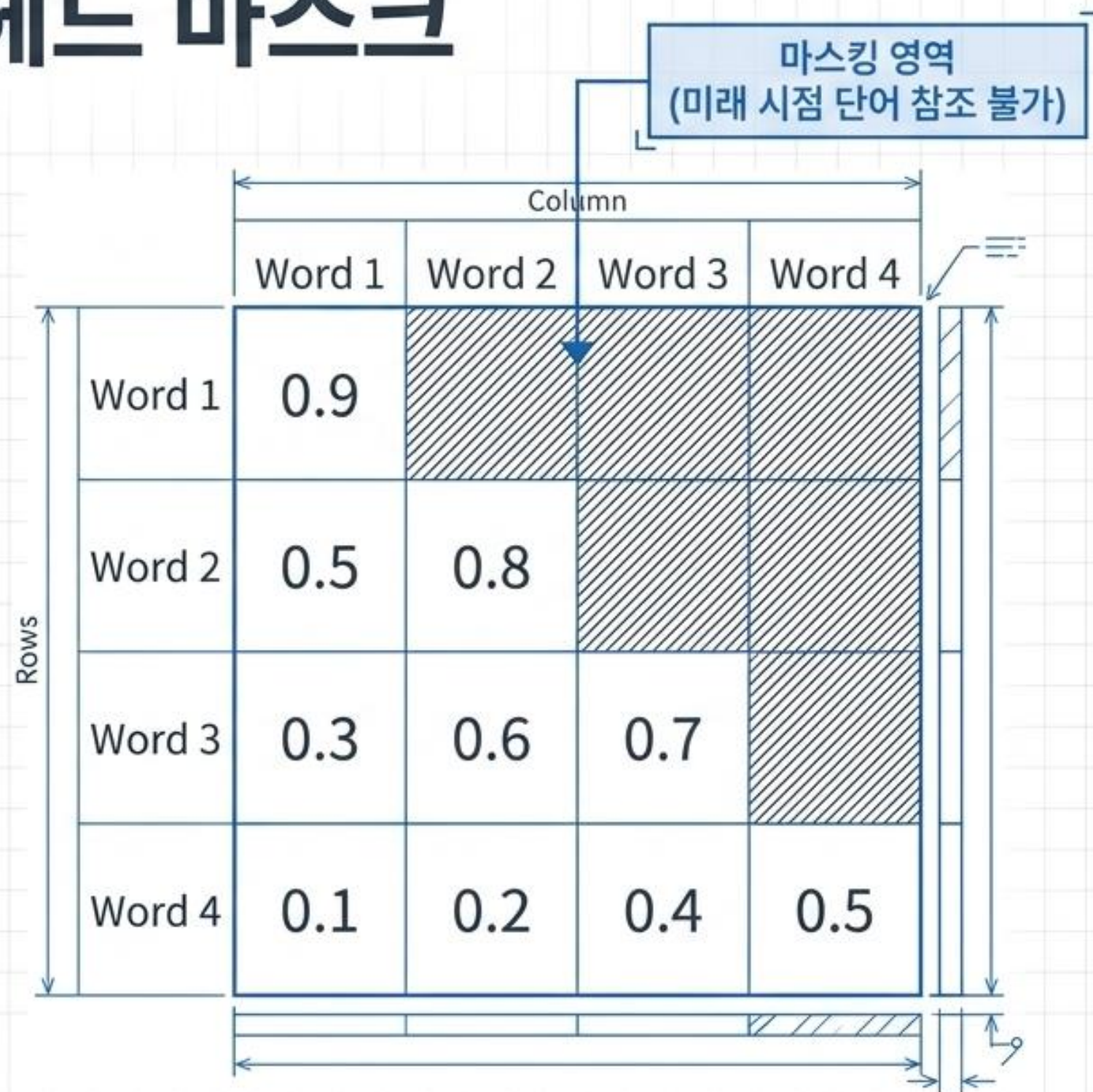
디코더로의 진입: 문장 행렬 입력과 교사 강요

- **정보 전달:** 6층의 인코더 연산이 모두 끝난 후, 완벽하게 압축된 최종 컨텍스트 행렬이 디코더로 전달됨
- **교사 강요 (Teacher Forcing):** 학습 시, 디코더는 정답에 해당하는 전체 문장(예: <sos> je suis étudiant)을 병렬 행렬 형태로 한 번에 입력받음
- **구조적 결함 발생:** RNN과 달리 병렬로 단어가 한 번에 들어오므로, 시퀀스를 예측할 때 '미래 시점의 단어'를 미리 보게 되는 커닝(Cheating) 가능성 존재



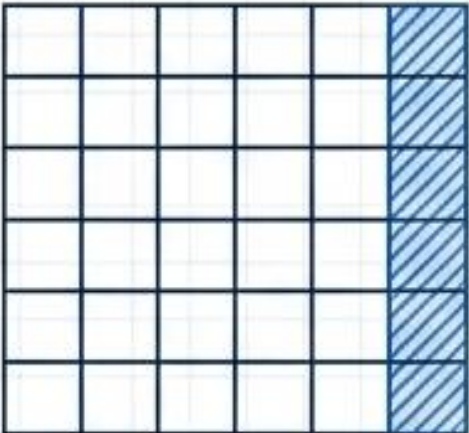
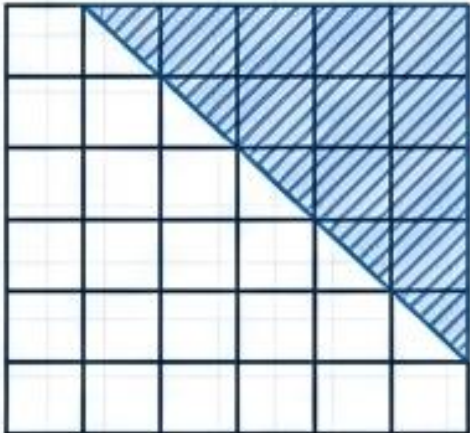
미래 단어 커닝 방지: 록-어헤드 마스크

- 구조적 위치: 디코더의 첫 번째 서브층(마스크드 셀프 어텐션) 내부에 적용
- 순차적 시뮬레이션: 디코더가 현재 시점의 단어를 예측할 때, 행렬 상 뒤에 위치한 미래의 단어들을 참고하지 못하도록 어텐션 스코어를 강제로 차단
- 마스크링 원리: 어텐션 스코어 행렬에서 대각선(자기 자신)을 기준으로 상단 우측(미래 시점)의 모든 영역을 차단 처리



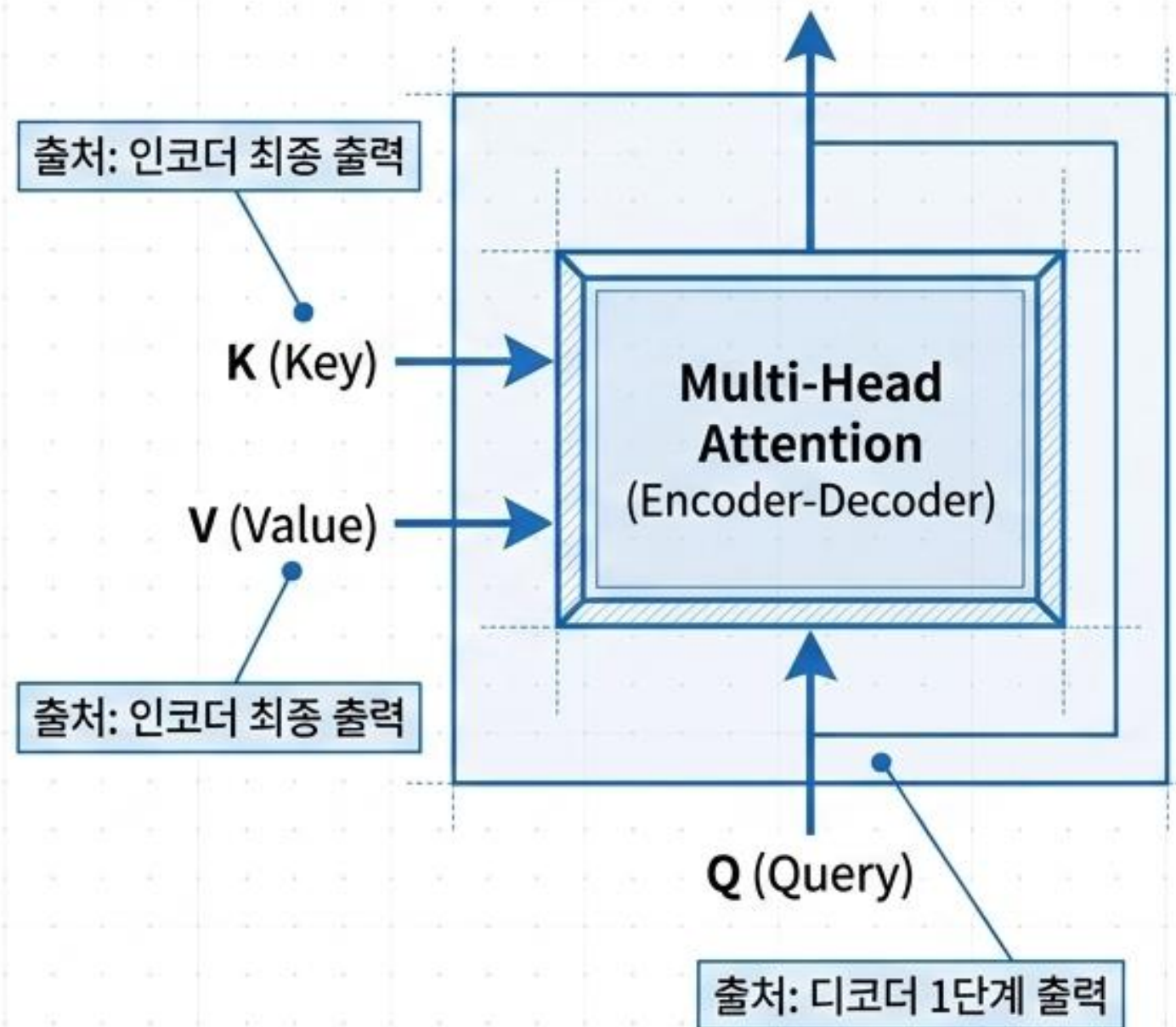
트랜스포머의 두 트랙 마스크 정책 비교

- ↳ 두 마스크 모두 내부적으로 '스케일드 닷-프로덕트 어텐션'의 스코어 연산 단계 직후에 적용됨
- ↳ 실제 시스템 구현 시, 디코더의 룩-어헤드 마스크는 패딩 마스크의 기능을 포함하도록 통합 설계됨

구분	패딩 마스크 (Padding Mask)	룩-어헤드 마스크 (Look-ahead Mask)
목적	길이를 맞추기 위한 <PAD> 토큰의 무의미한 연산 제외	학습 시 미래 시점의 정답 단어 커닝(Cheating) 원천 차단
적용 위치	인코더 셀프 어텐션 / 디코더 인코더-디코더 어텐션	디코더 마스크드 셀프 어텐션
마스크 형태	특정 열(Column) 전체 마스크 (세로줄)  Column Masking	대각선 기준 우상단 역삼각형 영역 마스크  Upper-Right Triangle Masking (Future Tokens)

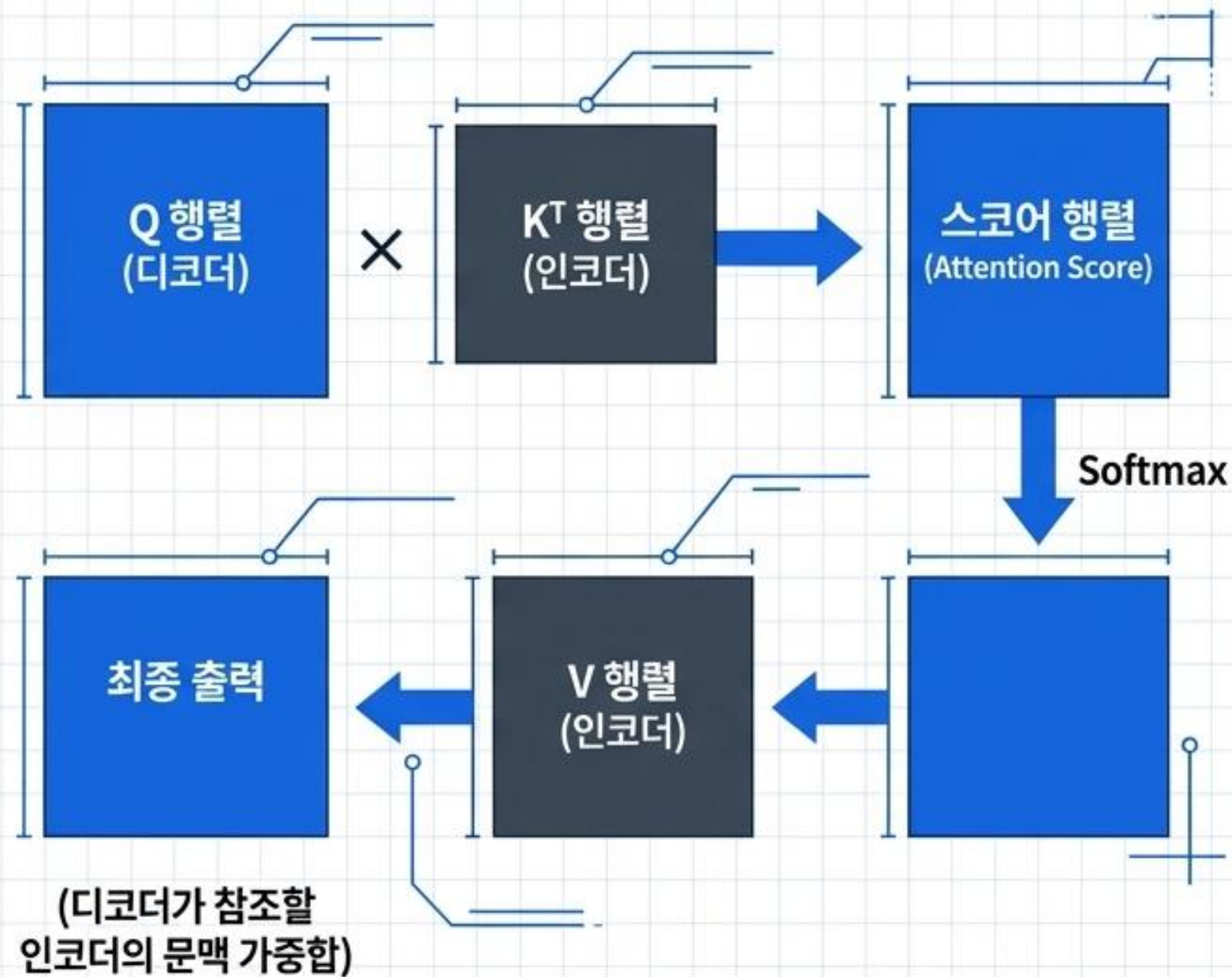
인코더-디코더 어텐션: 구조적 결합점

- **유일한 교차점**: 인코더의 압축된 정보와 디코더의 예측 과정이 물리적으로 만나는 핵심 서브층
- 이전 서브층들과 달리 **셀프 어텐션(Self-Attention)**이 **아님**. 연산 벡터의 출처가 서로 다르기 때문
- 출처 재확인:
 - Query (**Q**): 디코더의 첫 번째 서브층(마스크드 셀프 어텐션)을 통과한 결과 행렬
 - Key (**K**) & Value (**V**): 인코더의 최상단 층에서 계산이 완료되어 전달된 최종 출력 행렬



인코더-디코더 어텐션의 행렬 연산

- **내적 연산:** 디코더 단어 행렬(Q)과 인코더 단어 행렬(K) 간의 내적을 통해 어텐션 스코어 산출
- **최종 값 산출:** 산출된 확률 분포에 인코더의 단어 행렬(V)을 곱하여 최종 어텐션 값 도출
- **문맥 참조:** 이 연산을 통해 디코더는 번역할 단어를 생성할 때, 원본 입력 문장(인코더)의 어느 부분에 집중해야 할지 스스로 결정



데이터 여정의 완성: 트랜스포머 전체 파이프라인

- **1단계 (입력)**: 단어 임베딩 + 포지셔널 인코딩으로 공간적 맥락 확보
- **2단계 (인코딩)**: 셀프 어텐션과 잔차 연결을 통해 입력 문장의 완벽한 문맥적 압축
- **3단계 (디코딩)**: 룩-어헤드 마스크로 순차적 예측 환경을 시뮬레이션하며 인코더의 K, V 를 참조해 최적의 단어 추론
- **결론**: 압축(seq2seq)과 순차처리(RNN)의 병목을 버리고, 오직 병렬 어텐션만으로 자연어 처리의 새로운 패러다임 완성

